

Beyond the Syllabus

Multiplication and the FFT: A Solved Conjecture, an Open Problem, and a Dynamic
Closest Pair

Week 7 Supplement – TOAE209/TOAH209: Design and Analysis of Algorithms

Foreign Trade University

Outline

Why look beyond the syllabus?

- This week you proved two $\Theta(n \log n)$ bounds: the **closest pair of points** via the strip argument, and **polynomial multiplication** via the FFT (evaluate at roots of unity, multiply pointwise, interpolate back).
- You also saw Karatsuba's $\Theta(n^{\log_2 3})$ integer multiplication – Week 1's algorithm, revisited as one point on a whole *hierarchy* of multiplication algorithms.
- Three questions a curious student should ask next:
 - ① Karatsuba beat grade-school; the FFT-based approach beats Karatsuba. Is there an algorithm that beats *that*, reaching $O(n \log n)$ for integers too?
 - ② Is $\Theta(n \log n)$ actually **proven optimal** for the FFT itself, the way Week 3 proved Union-Find's bound optimal?
 - ③ Your closest-pair algorithm assumed a fixed, static point set. What if points keep arriving and leaving?

Note on scope

This deck deliberately does *not* revisit the matrix-multiplication exponent ω (Strassen, AlphaTensor/AlphaEvolve) – that story is already told in Week 2's supplement. Every claim below is sourced to a specific, checked paper.

Recap: the hierarchy you have built so far

Algorithm	Time	Where it comes from
Grade-school	$\Theta(n^2)$	direct definition of multiplication
Karatsuba (Week 1)	$\Theta(n^{\log_2 3}) \approx \Theta(n^{1.585})$	3 products instead of 4
Toom–Cook (k -way split)	$\Theta(n^{1+\varepsilon})$, any $\varepsilon > 0$	Karatsuba generalized to k pieces
Schönhage–Strassen (1971)	$O(n \log n \log \log n)$	FFT over integers (this week's idea)
Harvey–van der Hoeven	$O(n \log n)$	next slide

Each row is a strictly better asymptotic bound – and, as you will see, each is optimized for a *different* range of n in practice.

Schönhage & Strassen's 1971 conjecture, confirmed in 2021

Theorem (Harvey & van der Hoeven, *Annals of Mathematics* 193(2), 2021, pp. 563–617)

Two n -bit integers can be multiplied using $O(n \log n)$ bit operations. DOI:
10.4007/annals.2021.193.2.4.

- In 1971, Schönhage and Strassen introduced the $O(n \log n \log \log n)$ FFT-based algorithm you studied this week's ideas from, and **conjectured** $O(n \log n)$ was the truth – then it stood open for **48 years**.
- Harvey and van der Hoeven (2019, published 2021) finally constructed such an algorithm, confirming the conjecture.

The key idea: replace complex roots of unity with something exact

- This week's FFT multiplies coefficient vectors by evaluating at complex n -th roots of unity $\omega_n = e^{2\pi i/n}$. Done with floating-point arithmetic, this accumulates **rounding error** that grows with n – fine for polynomials with modest coefficients, fatal for multiplying huge exact integers.
- Schönhage–Strassen's fix: work in a ring $\mathbb{Z}/(2^m + 1)\mathbb{Z}$ that has an exact analogue of a root of unity (a power of 2), so the transform is **exact** integer arithmetic, not floating point – at the cost of an extra $\log \log n$ factor from recursively multiplying the (smaller) transformed coefficients.
- Harvey–van der Hoeven's contribution is a more intricate recursive choice of such rings (multi-dimensional FFTs over carefully engineered finite rings) that removes even that $\log \log n$ factor – full technical machinery is well beyond this course, but the *shape* of the idea – “do the FFT idea, but over an exact ring instead of $\mathbb{C}$ ” – is exactly this week's convolution trick, pushed one level further.

A cautionary crossover point

How large does n have to be?

Harvey and van der Hoeven's algorithm only beats Schönhage–Strassen (and Fürer's 2007 intermediate algorithm) once the integers being multiplied have more than

$2^{1729^{12}}$ bits.

- For scale: the number of particles in the observable universe is estimated at about 2^{270} . The crossover point is unimaginably larger than that.
- Source: D. Harvey, “Integer multiplication in time $O(n \log n)$ ” (talk slides, UNSW 2019); reported in *Communications of the ACM*, “Multiplication Hits the Speed Limit” (Jan. 2020).
- Compare Week 3's $\alpha(n)$ table: there, a function that is *unusably slow-growing in the good sense* (never exceeds 5 for any n you could write down). Here, a crossover point that is *unusably large in the bad sense*. Both are “ $O(1)$ for all practical purposes” – just in opposite directions.

What multiplication libraries actually run

- No production library implements Harvey–van der Hoeven: the crossover point above puts it firmly in “theoretical gold, not deployable” territory.
- GMP (used by Python’s `int`, SageMath, and most computer-algebra systems) instead switches, at *measured* thresholds tuned per CPU, between grade-school $\rightarrow$ Karatsuba $\rightarrow$ Toom–Cook $\rightarrow$ FFT/Schönhage–Strassen-style methods as n grows – exactly the hierarchy from two slides ago, chosen empirically rather than by asymptotics alone.
- Recent (2024) work continues to refine this in new directions, e.g. evaluating NTT (Number-Theoretic Transform, an exact finite-field FFT)/Karatsuba *hybrid* designs for $> 16,000$ -bit integers on FPGA hardware – the same idea this week’s FFT teaches you, retargeted at silicon.

The lesson

“Asymptotically optimal” and “fastest for the sizes you will ever multiply” are different claims. This week’s Karatsuba/FFT material is the one that actually wins in practice, for essentially every n a computer will ever see.

Recall Week 3's move: prove a matching lower bound

- For Union-Find, an upper bound ($O(m\alpha(n))$, Tarjan) was later matched by a lower bound (Fredman & Saks, 1989, cell-probe model) – so we *know* Union-Find is optimal.
- Natural question: is $\Theta(n \log n)$ for computing the Discrete Fourier Transform of n numbers similarly **proven optimal**?

Surprising answer: not in general

Whether every algorithm computing the DFT needs $\Omega(n \log n)$ operations, in a fully general model of computation, is a genuinely **open problem** in complexity theory. Unlike Union-Find, this loop has not been closed.

What *is* known

Unconditional lower bounds exist only for *restricted* models of computation:

- **Algebraic circuit model.** If you only count additions/multiplications in a linear circuit, $\Omega(n \log n)$ is known and matches the FFT's upper bound – so within that specific (very natural) model, the FFT is provably optimal.
- **Restricted unitary-gate model.** Ailon (2013), “A Lower Bound for Fourier Transform Computation in a Linear Model Over 2×2 Unitary Gates Using Matrix Entropy” (arXiv:1305.4745), proves $\Omega(n \log n)$ when each computation step is restricted to a 2×2 unitary operation on two coordinates at a time – a model chosen to mimic how real FFT circuits are built.
- **Online, cell-probe model.** Clifford & Jalsenius, “Tight Cell-Probe Bounds for Online Integer Multiplication and Convolution” (ICALP 2011, arXiv:1101.0768), prove a *matching* lower bound specifically for the *online* version (coefficients of one input revealed one at a time) – using the very same cell-probe technique as Fredman–Saks in Week 3.

No one has yet ruled out a fundamentally different, non-algebraic algorithm beating $n \log n$ for the general, offline DFT.

Recap: this week's static, single-shot algorithm

- This week's closest-pair algorithm assumes: (a) all n points are known in advance, and (b) you compute the answer once, sequentially.
- Real applications – ride-sharing dispatch, collision detection in a physics simulation, nearest-neighbor indexes – need the closest pair *as points keep being added and removed*, and ideally computed on many cores at once.

A parallel, batch-dynamic closest pair

Wang, Yu, Gu & Shun, “A Parallel Batch-Dynamic Data Structure for the Closest Pair Problem” (SoCG 2021; arXiv:2010.02379)

A data structure that maintains the closest pair of a point set under **batches** of insertions and deletions, with each batch update running in $O(\log^2 n)$ *parallel depth* (span) using near-linear total work, rather than recomputing the static $O(n \log n)$ algorithm from scratch after every change.

- The core reuse of this week’s idea: the same “strip near the dividing line” argument still bounds how much local structure a single update can disturb – it is generalized, not discarded.
- “Batch-dynamic” means: update b points at once in $O\left(\log^2 n + \frac{b}{p} \log^2 n\right)$ -style parallel time with p processors, rather than paying a full sequential pass per point.

Summary

- 1 **A 48-year conjecture, closed:** Harvey & van der Hoeven (2021) built an exact, $O(n \log n)$ integer-multiplication algorithm, confirming Schönhage & Strassen's 1971 guess – by pushing this week's “FFT over an exact ring instead of $\mathbb{C}$ ” idea further.
- 2 **Optimal in theory, irrelevant in practice (so far):** its crossover point, $2^{1729^{12}}$ bits, dwarfs any number a computer will ever multiply – real libraries still run tuned Karatsuba/Toom–Cook/Schönhage–Strassen hybrids.
- 3 **An open loop:** unlike Union-Find (Week 3), no one has proven a general $\Omega(n \log n)$ lower bound for the DFT – only for restricted models, using (among other tools) the same cell-probe technique from Week 3.
- 4 **Static → dynamic & parallel:** this week's closest-pair strip argument extends to a 2021 parallel batch-dynamic data structure, the same upgrade path Week 3's supplement traced for Union-Find/connectivity.

Looking ahead

Next week (Kleinberg & Tardos, Ch. 6) turns from divide-and-conquer's “split, recurse, combine” to **dynamic programming**'s “solve every subproblem once, reuse the answers” – a different way of taming an exponential search space.